

<i>Trường ĐH Công Thương TP.HCM</i> Khoa: CNTT Bộ môn: Kỹ thuật phần mềm LẬP TRÌNH MÃ NGUỒN MỞ	BÀI 2 LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP) TRONG PHP VÀ TÍCH HỢP FRONT-END Lập trình hướng đối tượng trong PHP	
---	---	---

A. MỤC TIÊU

- Hiểu và vận dụng được các khái niệm cơ bản của OOP: Class, Object, Property, Method.
- Thực hành tạo và sử dụng các thành phần OOP: Constructor, Destructor, Visibility, Static.
- Áp dụng tính kế thừa, đa hình, trừu tượng trong PHP.
- Sử dụng Interface và Abstract class để thiết kế hệ thống.
- Tổ chức mã nguồn với Namespace và Autoloading.

B. NỘI DUNG THỰC HÀNH

1. Tóm tắt lý thuyết

- Class & Object: Class là bản thiết kế, Object là thể hiện.
- Thuộc tính (Property) và Phương thức (Method).
- Phạm vi truy cập: public, private, protected.
- Constructor & Destructor.
- Static properties/methods.
- Kế thừa (extends), Đa hình.
- Abstract class và Interface.
- Namespace và Autoloading.

2. Bài tập tại lớp

Bài tập 01: Tạo class Car với các thuộc tính brand, color và phương thức showInfo() để hiển thị thông tin xe. Tạo 2 đối tượng từ class và gọi phương thức showInfo().

Hướng dẫn

```
<?php
class Car {
    public $brand;
    public $color;
```

```

        public function showInfo() {
            echo "Brand: {$this->brand}, Color: {$this->color}<br>";
        }
    }

    // Tạo đối tượng
    $car1 = new Car();
    $car1->brand = "Toyota";
    $car1->color = "Red";
    $car1->showInfo();

    $car2 = new Car();
    $car2->brand = "Honda";
    $car2->color = "Blue";
    $car2->showInfo();
    ?>

```

Bài tập 02: Tạo class Student với:

- Thuộc tính name, age (private)
- Constructor nhận 2 tham số để khởi tạo thuộc tính.
- Phương thức display() để hiển thị thông tin.
- Destructor in ra thông báo "Đối tượng đã bị hủy".
- Tạo 1 đối tượng và gọi các phương thức.

Hướng dẫn

```

<?php
class Student {
    private $name;
    private $age;

    public function __construct($name, $age) {
        $this->name = $name;
        $this->age = $age;
    }

    public function display() {
        echo "Name: {$this->name}, Age: {$this->age}<br>";
    }

    public function __destruct() {
        echo "Đối tượng Student đã bị hủy.<br>";
    }
}

```

```

    }
}

// Tạo đối tượng
$student1 = new Student("Nguyen Van A", 20);
$student1->display();
?>

```

Giải thích:

- `private` giới hạn truy cập bên ngoài class.
- `__construct` tự chạy khi tạo đối tượng.
- `__destruct` tự chạy khi đối tượng bị hủy.

Bài tập 03: Static và kế thừa

- Tạo class `MathHelper` với phương thức tĩnh `add($a, $b)` trả về tổng.
- Tạo class `AdvancedMath` kế thừa `MathHelper` và bổ sung phương thức tĩnh `pow($a, $b)` tính lũy thừa.
- Gọi các phương thức tĩnh mà không cần khởi tạo đối tượng.

Hướng dẫn

```

<?php
class MathHelper {
    public static function add($a, $b) {
        return $a + $b;
    }
}

class AdvancedMath extends MathHelper {
    public static function pow($a, $b) {
        return $a ** $b;
    }
}

// Gọi phương thức tĩnh
echo "3 + 5 = " . MathHelper::add(3, 5) . "<br>";
echo "2 ^ 3 = " . AdvancedMath::pow(2, 3) . "<br>";
?>

```

Giải thích:

- Phương thức static gọi trực tiếp bằng `TênClass::TênPhươngThức()`.
- Không cần tạo object.

Bài tập 04: Abstract class & Interface

- Tạo abstract class `Animal` có phương thức trừu tượng `makeSound()`.
- Tạo 2 class `Dog` và `Cat` kế thừa `Animal` và triển khai `makeSound()`.
- Tạo interface `CanRun` với phương thức `run()`.
- `Dog` triển khai `CanRun` và in "Dog is running...".

Hướng dẫn

```
<?php
abstract class Animal {
    abstract public function makeSound();
}

class Dog extends Animal implements CanRun {
    public function makeSound() {
        echo "Woof! Woof!<br>";
    }

    public function run() {
        echo "Dog is running...<br>";
    }
}

class Cat extends Animal {
    public function makeSound() {
        echo "Meow! Meow!<br>";
    }
}

interface CanRun {
    public function run();
}

// Sử dụng
$dog = new Dog();
$dog->makeSound();
$dog->run();

$cat = new Cat();
$cat->makeSound();
?>
```

Giải thích:

- Abstract class: chứa phương thức trừu tượng chưa có nội dung.
- Interface: chỉ định phương thức phải triển khai.
- implements dùng để cài đặt interface.

Bài tập 05: Namespace và Autoloading

- Tạo thư mục *app/Models* chứa class User (namespace App\Models) với phương thức sayHello().
- Tạo file *autoload.php* dùng spl_autoload_register() để tự động nạp class khi cần.
- Trong file *index.php*, gọi sayHello() của class User mà không require thủ công.

Hướng dẫn

Cấu trúc thư mục:

```
project/
  app/
    Models/
      User.php
  autoload.php
  index.php
```

app/Models/User.php

```
<?php
namespace App\Models;

class User {
    public function sayHello() {
        echo "Hello from User class!<br>";
    }
}
```

autoload.php

```
<?php
spl_autoload_register(function ($class) {
    $path = str_replace("\\", "/", $class) . ".php";
    if (file_exists($path)) {
        require $path;
    }
});
```

index.php

```
<?php
require 'autoload.php';
```

```
use App\Models\User;

$user = new User();
$user->sayHello();

?>
```

Giải thích:

- namespace dùng để phân vùng class, tránh trùng tên.
- `spl_autoload_register()` tự động nạp file khi cần, không phải require thủ công.
- `use` để gọi namespace thuận tiện hơn.

3. Bài tập ôn luyện

Bài tập 06: Viết class `BankAccount` với các thuộc tính: số tài khoản, tên chủ tài khoản, số dư. Thực hiện các phương thức: nạp tiền, rút tiền (kiểm tra số dư), hiển thị số dư.

Bài tập 07: Viết hệ thống quản lý thư viện:

- Book (title, author, price)
- Ebook kế thừa Book và có thêm `fileSize`
- Interface `Downloadable` với phương thức `download()`
- Ebook triển khai `Downloadable`

Bài tập 08: Viết ứng dụng quản lý sinh viên:

- Class `Person` (name, age)
- Class `Student` kế thừa `Person` (studentID)
- Namespace `App\Students`
- Autoloading với `spl_autoload_register`.